Herald

Herald — `commons_tls` Module Guide (Operator / Developer)

Field	Value
Revision	1
Created	2026-05-31
Last modified	2026-05-31
Status	active
Status summary	Nano-detail per-module reference for `commons_tls` — Herald's TLS cert-sourcing helper (Wave 4a). Documents the two exported entry points `LoadOrGenerate(certPath, keyPath)` and `ResolveCertSource(cfg Config)`, the `Config`/`CertSource` types, the flag→env→dev-autogen precedence ladder, the prod-mode fail-loud override (`HERALD_AUTH_MODE=jwks`), the ECDSA P-256 self-signed dev-cert generator (SAN `[localhost, 127.0.0.1, ::1]`, 365-day validity, key persisted 0600), and HOW serving flavors feed the resolved `*tls.Certificate` into the dual HTTP/3 (quic-go) + TLS 1.3 listener and the Alt-Svc header. ANTI-BLUFF: every section documents only what the source under `commons_tls/` (and its `commons/cli/` consumers) actually does as of this revision.
Issues	(none specific to this guide)
Continuation	bump when `commons_tls` gains cert rotation / reload-on-SIGHUP, a CA-signed-chain validation helper, or when the production `--tls-cert` path lands operator-supplied live evidence under `docs/qa/<run-id>/`.

Table of contents

§1. Overview

`commons_tls` (Go package `commons_tls`, module path `github.com/vasic-digital/herald/commons_tls`) is Herald's TLS cert-sourcing helper. It answers exactly one question for the serving flavors: **"what `*tls.Certificate` should this listener present, and where did it come from?"** It does NOT bind any socket or run any handshake itself — it only *sources* the certificate; the dual-listener orchestration lives in `commons/cli/serve.go` (see §5).

It exposes **two** entry points (both documented in the package doc at the top of `cert.go`):

1. `LoadOrGenerate(certPath, keyPath)` — the low-level primitive. Loads a pair if BOTH files exist; generates a fresh ECDSA P-256 self-signed pair if NEITHER exists; errors on an asymmetric (exactly-one-present) configuration.
2. `ResolveCertSource(cfg Config)` — the full Herald cert-resolution policy. Walks a precedence ladder (operator flags → env vars → dev-autogen), with a hard prod-mode fail-loud override, and returns a `CertSource` that tags *where* the cert came from.

The module is **pure standard library** — `crypto/ecdsa`, `crypto/x509`, `crypto/tls`, `encoding/pem`, `os`, etc. There is no third-party dependency in `commons_tls/go.mod` (the only `require` is the Go version `go 1.25.3`). The quic-go / HTTP/3 plumbing it feeds lives in its consumers (`commons/cli/`), not here.

Wave 4a context: this module is the cert substrate for the dual HTTP/3 + TLS 1.3 listener. Catalogue-Check (§11.4.74) was run with a no-match result, so it is vendored as a Herald-internal package (evidence: `docs/catalogue-checks/HRD-100-commons-tls.md`, cited in `go.mod`).

§2. The API

The entire public surface is two functions and two types.

§2.1 `Config`

```
type Config struct {
    CertPath string // operator --tls-cert path; empty ⇒ fall to
        env tier
    KeyPath  string // operator --tls-key path; empty ⇒ fall to
        env tier
    ProdMode bool   // true ⇒ refuse dev-autogen, fail loud with
        no explicit cert
}
```

`Config` is the input to `ResolveCertSource`. When both `CertPath` and `KeyPath` are set, the pair is loaded directly (Tier 1). When both are empty, the env vars are consulted (Tier 2). `ProdMode` is the production signal — Herald convention is `ProdMode = (os.Getenv("HERALD_AUTH_MODE") == "jwks")`, wired in `serve.go`.

§2.2 `CertSource`

```
type CertSource struct {
    Cert   *tls.Certificate
    Source string // "flag" | "env" | "autogen-dev"
}
```

`Source` is a provenance string — `"flag"` when both `Config.CertPath` + `Config.KeyPath` were supplied, `"env"` when both `HERALD_TLS_CERT_PATH` + `HERALD_TLS_KEY_PATH` were set, and `"autogen-dev"` when the cert was loaded from `~/.herald/dev-{cert,key}.pem` (generated on first call, reused thereafter).

§2.3 Functions

Function	Signature	Behaviour
`LoadOrGenerate`	`LoadOrGenerate(certPath, keyPath string) (*tls.Certificate, error)`	BOTH files exist → `tls.LoadX509KeyPair` and return. NEITHER exists → generate a fresh ECDSA P-256 self-signed pair (SAN `[localhost, 127.0.0.1, ::1]`, 365-day validity), persist cert at `0644` + key at `0600`, return the loaded pair. EXACTLY ONE exists → error (asymmetric-config guard, see §4).
`ResolveCertSource`	`ResolveCertSource(cfg Config) (*CertSource, error)`	Runs the flag→env→dev-autogen precedence ladder with the prod-mode fail-loud override (see §3). Returns a `*CertSource` tagged with the originating tier.

There are no other exported symbols. `devSANs` and `devValidity` are package-private constants (`[]string{"localhost","127.0.0.1","::1"}` and `365*24*time.Hour` respectively).

§3. Cert resolution policy (flag → env → dev-autogen, prod fail-loud)

`ResolveCertSource` is the load-bearing policy. Precedence, high → low:

1. **Tier 1 — operator flags.** If `cfg.CertPath != "" && cfg.KeyPath != ""`, load that pair and return `Source: "flag"`. If exactly ONE of the two is set, return an error (`--tls-cert + --tls-key must be supplied together`) — Herald refuses to silently fall through to a lower tier when the operator clearly intended to supply a flag-tier cert but typo'd one half.
2. **Tier 2 — operator env.** If both `HERALD_TLS_CERT_PATH` and `HERALD_TLS_KEY_PATH` are set, load that pair and return `Source: "env"`. Again,

exactly-one-set is an error (`HERALD_TLS_CERT_PATH + HERALD_TLS_KEY_PATH must be supplied together`).

3. **Prod-mode gate (checked BEFORE Tier 3).** If `cfg.ProdMode` is true and neither flags nor env supplied a cert, return an error — there is **NO silent dev-autogen fallback in production**. The error text contains the substring `production mode (HERALD_AUTH_MODE=jwks) requires --tls-cert ...`.
4. **Tier 3 — dev auto-generation.** In dev mode only, resolve `$HOME` and call `LoadOrGenerate($HOME/.herald/dev-cert.pem, $HOME/.herald/dev-key.pem)`, returning `Source: "autogen-dev"`. First call generates, subsequent calls reuse.

Precedence is real, not just label-correct. `TestResolveCertSource_FlagWinsOverEnv` seeds two *distinct* pairs, sets the env to the env pair, supplies the flag pair, and asserts the resolved cert bytes match the **flag** file byte-for-byte — proving flag actually wins, not merely that the `Source` label says `"flag"`.

§4. The dev self-signed cert generator

When `LoadOrGenerate` finds NEITHER file on disk, it generates a fresh dev cert with these exact properties (all asserted by `TestLoadOrGenerate_NeitherExists_AutoGenerates`):

- **Key:** `ecdsa.GenerateKey(elliptic.P256(), …)` — ECDSA on curve **P-256**.
- **Signature algorithm:** `x509.ECDSAWithSHA256`.
- **SAN list:** `localhost` (DNS), `127.0.0.1` (IP), `::1` (IP) — loopback testing on dual-stack IPv4 + IPv6 hosts plus name resolution. IPs are routed to `IPAddresses`, names to `DNSNames`, by `net.ParseIP`.
- **Validity:** `NotBefore = now − 1min` (clock-skew slack), `NotAfter = now + 365 days` (`devValidity`).
- **Key usage:** `DigitalSignature | KeyEncipherment`; ExtKeyUsage `ServerAuth | ClientAuth`.
- **Subject org:** `"Herald DEV (commons_tls auto-gen)"` — a visible marker that this is a dev cert, not a production CA-signed chain.
- **Persistence:** the destination directory is `MkdirAll`'d at `0700`; the cert is written at `0644` (public material); the key is written at `0600` and then **defensively re-`chmod`'d to `0600`** in case the process umask widened the `OpenFile` mode. The 0600 key mode is asserted by the test.

§4.1 The asymmetric-config guard

If EXACTLY ONE of cert/key exists on disk, `LoadOrGenerate` returns an error whose text contains `asymmetric` (asserted by `TestLoadOrGenerate_OnlyOneExists_Errors`, both directions). Rationale, verbatim from the source: Herald refuses to auto-pair operator-supplied material with auto-generated material because the asymmetric state is almost certainly an operator typo, and silently auto-pairing would bind a key the operator did not authorize. This is a security decision, not a convenience guard.

§5. How serving flavors use it — HTTPS + HTTP/3 + Alt-Svc

The consumer is `commons/cli/serve.go` (the shared dual-listener orchestration that every serving flavor's `serve` subcommand builds on). The flow:

1. **Mode signals.** `serve.go` reads `HERALD_DISABLE_HTTP3=1` (the UDP-blocked-environment escape hatch → TCP-only) and sets `opts.ProdMode = (os.Getenv("HERALD_AUTH_MODE") == "jwks")`.
2. **Resolve the cert.** `serve.go` calls `commons_tls.ResolveCertSource(commons_tls.Config{CertPath: …, KeyPath: …, ProdMode: opts.ProdMode})` and takes `src.Cert` — a single `*tls.Certificate` shared by BOTH listeners. In the HTTP/3-enabled path the cert is mandatory; in the TCP-only path it is resolved too (so TCP can run HTTPS when a cert is available, plaintext otherwise for backward compat).
3. **HTTP/3 (QUIC) listener.** `startQUIC(ctx, addr, handler, cert)` (`commons/cli/h3.go`) builds a `digital.vasic.http3/pkg/server.Server` with a `tls.Config{Certificates: []tls.Certificate{*cert}, MinVersion: tls.VersionTLS13, NextProtos: ["h3"]}`. HTTP/3 **requires TLS 1.3 on the wire** (RFC 9114) and uses the `h3` ALPN identifier — both are set here from the same `commons_tls`-resolved cert. `startQUIC` rejects a `nil` cert or an empty `cert.Certificate` chain before binding.
4. **TCP (HTTPS + H2) listener.** When `cert != nil`, `serve.go` sets `tcpSrv.TLSConfig = &tls.Config{Certificates: []tls.Certificate{*cert}}` so the TCP listener serves HTTPS + HTTP/2 on the same cert. So both protocols speak HTTPS off ONE `commons_tls` resolution.
5. **Alt-Svc advertisement.** `AltSvcMiddleware(h3Port)` (`commons/cli/altsvc.go`) is injected into the Gin chain. It emits `Alt-Svc: h3=":<h3Port>"; ma=2592000`, telling Alt-Svc-aware clients (curl `--http3`, Chromium, Firefox) to upgrade subsequent requests to the QUIC listener `startQUIC` bound. When `h3Port <= 0` (HTTP/3 disabled) the middleware is a no-op and emits NO header — advertising a port that isn't listening is worse than not advertising.

So `commons_tls` is the single seam that decides the certificate identity; `commons/cli/` decides how that certificate is presented across the two protocols.

§6. Operator section — production certs vs the dev self-signed path

§6.1 Dev / local — let Herald auto-generate

Do nothing. Run the flavor in dev mode (do NOT set `HERALD_AUTH_MODE=jwks`, do NOT pass `--tls-cert`/`--tls-key`). On first serve, Herald generates and persists:

- `~/.herald/dev-cert.pem` (mode 0644)
- `~/.herald/dev-key.pem` (mode 0600)

These are self-signed for `localhost` / `127.0.0.1` / `::1`, valid 365 days. Clients must skip verification for self-signed certs (`curl -k`, `curl --http3 -k`). The `Source` reported is `autogen-dev`.

To renew / rotate the dev cert: delete both `~/.herald/dev-{cert,key}.pem` and restart the flavor — the next serve regenerates them. (Deleting only ONE triggers the §4.1 asymmetric-config error — delete both or neither.)

§6.2 Production — supply your own CA-signed cert

In production you MUST supply a real cert (the dev-autogen path is hard-disabled when `HERALD_AUTH_MODE=jwks`). Two ways, flag-tier wins over env-tier:

Tier 1 — flags:

```
HERALD_AUTH_MODE=jwks <flavor>herald serve \
  --tls-cert /etc/herald/tls/fullchain.pem \
  --tls-key  /etc/herald/tls/privkey.pem
```

Tier 2 — env:

```
export HERALD_AUTH_MODE=jwks
export HERALD_TLS_CERT_PATH=/etc/herald/tls/fullchain.pem
export HERALD_TLS_KEY_PATH=/etc/herald/tls/privkey.pem
<flavor>herald serve
```

Operator notes:

- Both halves of a tier MUST be supplied together. `--tls-cert` without `--tls-key` (or one env var without the other) is a hard error — Herald will not fall through to a lower tier.
- The cert file should be the **full chain** (leaf + intermediates) so HTTP/3 and HTTPS clients can build a valid path; the key file is the matching private key. PEM-encoded, as `tls.LoadX509KeyPair` expects.
- Restrict the key file to `0600` (or tighter) and an operator-owned path; Herald enforces `0600` on the dev key but trusts your perms on operator-supplied keys.
- HTTP/3 needs the UDP port reachable. If your environment blocks UDP, set `HERALD_DISABLE_HTTP3=1` to bind TCP-only (HTTPS+H2); the Alt-Svc header is then suppressed automatically.

§7. Troubleshooting (cert errors)

Symptom / error text	Cause	Fix
`commons_tls: asymmetric cert/key configuration — <X> exists but <Y> does not`	Only ONE of the dev pair (or a supplied pair) is on disk.	Supply BOTH files, or (dev) delete BOTH `~/.herald/dev-{cert,key}.pem` and let Herald regenerate.
`commons_tls: production mode (HERALD_AUTH_MODE=jwks) requires --tls-cert ...`	`HERALD_AUTH_MODE=jwks` set but no flag/env cert supplied.	Supply `--tls-cert`/`--tls-key` or `HERALD_TLS_CERT_PATH`/`HERALD_TLS_KEY_PATH` (§6.2), or unset `HERALD_AUTH_MODE` for dev.
`--tls-cert + --tls-key must be supplied together`	Exactly one flag set.	Pass both flags.

Symptom / error text	Cause	Fix
`HERALD_TLS_CERT_PATH + HERALD_TLS_KEY_PATH must be supplied together`	Exactly one env var set.	Export both env vars.
`commons_tls: load existing cert/key pair: .../load flag-supplied cert/key: ...`	The PEM files exist but are malformed, mismatched (key not for cert), or wrong format.	Verify with `openssl x509 -in cert.pem -noout -text` and `openssl ec -in key.pem -noout`; ensure the key matches the cert.
`h3: cert is nil` / `h3: cert.Certificate chain is empty` (from `startQUIC`)	HTTP/3 enabled but no usable cert reached the listener.	Ensure cert resolution succeeded; in production supply a cert; in dev confirm `~/.herald` is writable.
Client `x509: certificate signed by unknown authority` against a dev cert	The dev cert is self-signed.	Use `-k` / skip-verify for local testing, or supply a CA-signed cert (§6.2) for any non-loopback client.
Client `x509: certificate is valid for localhost, 127.0.0.1, ::1, not <host>`	Connecting to the dev cert via a non-loopback hostname.	Connect via `localhost`/`127.0.0.1`/`::1`, or supply a production cert whose SAN covers your hostname.
HTTP/3 never engages (clients stay on H2)	UDP blocked, or `HERALD_DISABLE_HTTP3=1`, or `h3Port <= 0` (Alt-Svc suppressed).	Open the UDP port; confirm `HERALD_DISABLE_HTTP3` is unset; check the Alt-Svc header is present (`curl -sI https://host:port`).

§8. Testing notes

Tests live in `commons_tls/cert_test.go` and run with no external services (real temp files under `t.TempDir()`, `$HOME` redirected via `t.Setenv`):

```
go test -race -count=1 ./commons_tls/...
```

Test	Proves
`TestLoadOrGenerate_BothFilesExist_LoadsThem`	A second call with both files present LOADS (cert bytes byte-identical), does not re-generate.
`TestLoadOrGenerate_NeitherExists_AutoGenerates`	Generates a real ECDSA P-256 cert: parses the leaf, asserts `ECDSAWithSHA256`, curve `P-256`, SAN `[localhost, 127.0.0.1, ::1]`, key file mode `0600`.
`TestLoadOrGenerate_OnlyOneExists_Errors`	The asymmetric-config guard fires in BOTH

Test	Proves
	directions (only-cert, only-key); error text contains `asymmetric`.
`TestResolveCertSource_DevModeNoFlags_AutoGen`	Dev mode + no flags + no env → `Source == "autogen-dev"`, files persisted under `$HOME/.herald/`.
`TestResolveCertSource_ProdModeNoFlags_Errors`	Prod mode + no cert → error containing `production mode` (proves the prod branch actually fired).
`TestResolveCertSource_FlagWinsOverEnv`	Flag tier beats env tier — resolved cert bytes match the FLAG file byte-for-byte (precedence is real).

Anti-bluff observations worth preserving when editing these tests (per §107): the generator test parses the actual `x509.Certificate` and asserts cryptographic properties (algorithm, curve, SAN) rather than "a file appeared"; the key-mode test stats the file and asserts `0600`; the flag-wins test cross-checks the resolved bytes against the flag file so a label-only pass cannot hide a wrong-tier load. The HTTP/3 consumer side (`commons/cli/h3_test.go` `TestStartQUIC_RealClientRoundTrip`, `commons/cli/altsvc_test.go`) carries the load-bearing handshake + exact-header evidence that the `commons_tls`-resolved cert is actually served.

§9. References

- Source: `commons_tls/cert.go` and `commons_tls/cert_test.go`.
- Package doc: the comment block at the top of `cert.go` (the two-entry-point + §107 anti-bluff rationale).
- Consumers (dual listener): `commons/cli/serve.go` (resolution + dual-listener orchestration), `commons/cli/h3.go` (`startQUIC` — TLS 1.3 + `h3` ALPN QUIC listener), `commons/cli/altsvc.go` (`AltSvcMiddleware` — the `Alt-Svc: h3=...` header).
- HTTP/3 dependency: `digital.vasic.http3/pkg/server` — vendored as the `http3` git submodule under `submodules/http3` (referenced via a `replace` directive in `commons/go.mod`, not `go.work`).
- Catalogue-Check evidence: `docs/catalogue-checks/HRD-100-commons-tls.md` (§11.4.74 no-match → vendored Herald-internal).
- Spec: `docs/specs/mvp/specification.V4.md` §11 (Channel + transport substrate) and the Wave 4a transport upgrades recorded in `CLAUDE.md`.
- Related module guides: `docs/guides/COMMONS_WATCH.md`, `docs/guides/COMMONS_WORKABLE.md`.

Sources verified

This guide documents internal Herald source only; no external service/library online documentation was relied on. All behavioural claims are grounded in the cited source files as of 2026-05-31.

Verified 2026-05-31: internal doc — no external online sources. Behavioural claims derive from `commons_tls/cert.go` + `commons_tls/cert_test.go` and the consumer files `commons/cli/serve.go`, `commons/cli/h3.go`, `commons/cli/altsvc.go` (all read 2026-05-31). `commons_tls` has NO third-party dependency (`commons_tls/go.mod` requires only `go 1.25.3`); the HTTP/3 listener it feeds uses the vendored `digital.vasic.http3` submodule. Protocol facts cited (HTTP/3 requires TLS 1.3, `h3` ALPN, RFC 9114; Alt-Svc `ma` RFC 7838) are restated from the source comments in `commons/cli/h3.go` and `commons/cli/altsvc.go`. Re-verify on a `commons_tls` API change or an `http3` submodule major bump.